

SoftRole-HetNet: Implementing It in the HetNet Repo

Sep 28, 2026 · @Akshat Guduru

Summary

SoftRole-HetNet fits into your fork as one new package, `softrole/` (628 lines), plus 20 changed lines in `main.py` and `hetnet_ext/recording.py`. Nothing under `hetgat/`, `trainer.py`, `multi_processing.py` or `envs/` changes, so HetNet stays exactly the baseline it is today. Every equation referenced here is from the companion guide, *SoftRole-HetNet: The Architecture, Explained*.

I built this in a copy of your fork and ran it (PyTorch 2.2.1, DGL 2.1.0, CPU, float64 as `main.py` sets). The code below is excerpted from that run, not written from memory:

- **18 new unit tests pass**, one per property the architecture guide claims (section 9). The fork's 29 existing tests still pass.
- **Your gradient-contract probe passes on SoftRole unchanged**: 4 processes, 3 updates, 51 parameter tensors, maximum gradient error 0.
- **The HetNet path is bit-identical**. HetNet-Binary trained through the patched `main.py` gave the same metrics and the same parameter signatures as the original `main.py`, for the same seed.
- **Training runs end to end through `main.py`**: single-process and 2-process (spawn), on the fixed PCP team, on teams resampled every episode, and with a mid-episode sensor loss.

Principles

Principle	Reason
Add, never edit, HetNet's code	The baseline must be the code behind the published numbers; a shared refactor can change it silently
Inherit HetNet's optimizer pipeline unchanged	Batching, gradient ÷ steps, clipping at 0.75 and RMSprop are then identical, so they cannot explain a difference

Principle	Reason
One test per claimed equation	A property that is only asserted in prose is not yet a property of the code
Every baseline is a flag on the same network	Ablations then have exactly the same parameter count, 233,513, so capacity is matched by construction

What changes

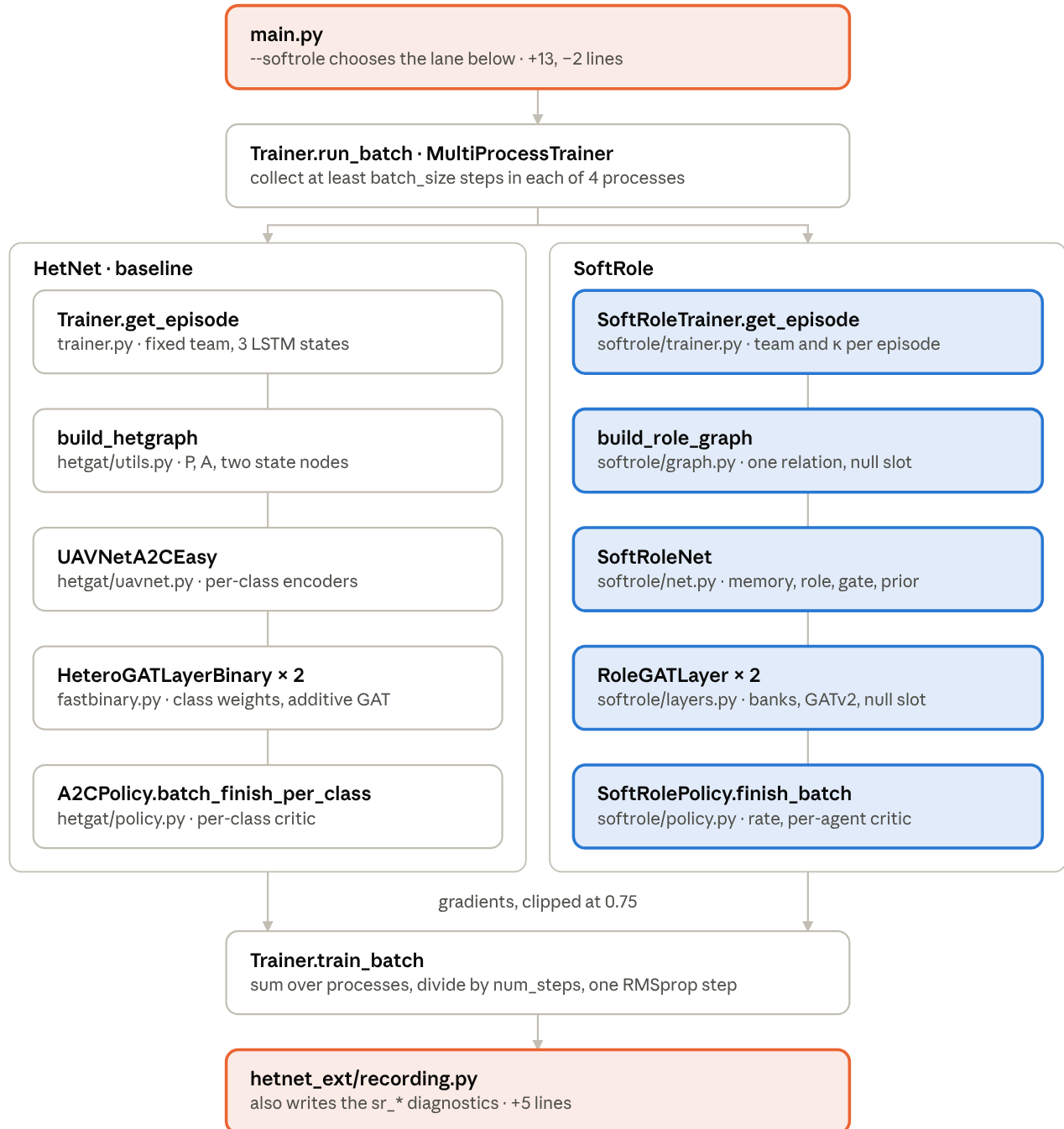
File	Change	Lines
<code>softrole/</code> (9 modules)	New: graph, bank, layer, network, policy, trainer, environment helpers, flags, evaluation	+628
<code>tests/test_softrole.py</code> , <code>tests/helpers/softrole_gradient_probe.py</code>	New tests	+306
<code>main.py</code>	Register flags; build <code>SoftRolePolicy</code> ; choose <code>SoftRoleTrainer</code>	+13, -2
<code>hetnet_ext/recording.py</code>	Also record <code>sr_*</code> diagnostics	+5
<code>hetgat/*</code> , <code>trainer.py</code> , <code>multi_processing.py</code> , <code>action_utils.py</code> , <code>envs/*</code>	Untouched	0

The complete patch is attached to the conversation (`softrole.patch` , one commit on top of your fork's HEAD).

At a glance

SoftRole swaps the middle lane; the loop around it stays HetNet code

☐ new file ☒ changed file ☐ untouched



Code path per update · HetNet and SoftRole lanes, shared loop and optimizer

Read top to bottom. Both methods enter through the same `main.py`, collect episodes with the same `run_batch`, and finish in the same `train_batch`. Only the five calls in the middle differ, and on the SoftRole side each is a new file standing beside the HetNet file it replaces.

1. Repository layout and flags

`--softrole` selects the new model; without it `main.py` runs HetNet exactly as before. The package mirrors HetNet's own split of responsibilities, one module per HetNet file it stands beside:

New module	Stands beside	Holds
<code>softrole/graph.py</code>	<code>hetgat/utils.py</code> · <code>build_hetgraph</code>	Per-step graph from true positions, with null slots and one SEN
<code>softrole/bank.py</code>	(none)	The weight bank $W(g) = \sum_c g_c W_c$
<code>softrole/layers.py</code>	<code>hetgat/graph/fastbinary.py</code>	One communication round: channel, decoding, GATv2, null slot, SEN
<code>softrole/net.py</code>	<code>hetgat/uavnet.py</code> · <code>UAVNetA2CEasy</code>	Memory, role, gate, rounds, masked policy, critic, prior
<code>softrole/policy.py</code>	<code>hetgat/policy.py</code> · <code>A2CPolicy</code>	Rollout buffers, GAE, the loss, gradient routing
<code>softrole/trainer.py</code>	<code>trainer.py</code> · <code>Trainer</code>	A subclass that replaces only <code>get_episode</code> and <code>compute_grad</code>
<code>softrole/env.py</code>	<code>envs/.../predator_capture_env.py</code>	Team composition, κ , true positions, sensor masking; the env class is untouched

New module	Stands beside	Holds
<code>softrole/evaluate.py</code>	<code>eval_trainer.py</code>	Frozen-policy evaluation with events and interventions
<code>softrole/args.py</code>	<code>main.py</code> parser	The flags below

The change to `main.py`

```
+from softrole.args import add_softrole_args, resolve_softrole_args
+from softrole.policy import SoftRolePolicy
+from softrole.trainer import SoftRoleTrainer
...
+add_softrole_args(parser)
  resolved_seed = seed_everything(parser.parse_known_args()[0].seed)
...
  args.nfriendly = args.nfriendly_P + args.nfriendly_A
+if args.softrole:
+    resolve_softrole_args(args)
...
  if args.commnet:
      policy_net = CommNetMLP(args, num_inputs, 4)
+elif args.softrole:
+    policy = SoftRolePolicy.from_args(args)
+    policy_net = policy.model
  elif args.hetgat:
      ...
-    lambda: Trainer(args, policy_net, data.init(args.env_name, args),
policy))
+    lambda: (SoftRoleTrainer if args.softrole else Trainer)(
+        args, policy_net, data.init(args.env_name, args), policy))
```

Registering flags draws no random numbers, so `seed_everything` sees the same stream as before; that is why the HetNet run stays bit-identical. `resolve_softrole_args` sets `hetgat` and `hetgat_a2c` so the inherited `run_batch` and `train_batch` take HetNet's branches.

Flags

Flag	Symbol	Default	Meaning
<code>--softrole</code>		off	Train SoftRole-HetNet
<code>--sr_role</code>		stochastic	stochastic (full model), deterministic ($\rho = \mu(H, \kappa)$), constant ($\rho = 0$), capability ($\rho = \mu(\kappa)$)
<code>--sr_experts</code>	K	4	Weight sets per bank
<code>--sr_role_dim</code>	d _p	3	Must equal K – 1 (softmax shift invariance); checked at start-up
<code>--sr_beta</code>	β	0	Weight of the rate term; swept
<code>--sr_lambda_rho</code>	λ_p	0.9	Prior persistence
<code>--sr_var_floor</code>	$\sigma^2_{\min}$	0.01	Variance floor of q
<code>--sr_pre_dim, --sr_hid_dim</code>		128, 64	Observation encoder and LSTM widths
<code>--sr_comm_range</code>		–1	Euclidean range; –1 connects all pairs, as HetNet’s default does
<code>--sr_p_range, --sr_a_range</code>		fixed	Team sizes sampled per episode; default is HetNet’s fixed team
<code>--sr_sensor_loss_step</code>		–1	Step at which one predator loses its sensor
<code>--msg_dim, --gamma, --detach_gap, --lrate</code>	n, γ , BPTT window	HetNet’s	Shared with HetNet, so both methods read the same values

The four `--sr_role` values are the baselines of the claim: full model, CASH-style direct conditioning through the same channel, shared weights plus κ , and capability-only gating. They run the same network with the same 233,513 parameters and differ only in how p is

formed.

2. Graph construction

HetNet builds four agent relations and two state nodes whose shape is fixed by the class counts. SoftRole builds one relation over whoever is active, adds a null edge into every receiver, and keeps one SEN:

$$\begin{aligned} \mathcal{E}_t = \big\{ \langle k \rightarrow j \rangle; \langle j, k \rangle \in \mathcal{N}_t, \langle k \neq j \rangle; \\ \bigvee p_k - p_j \wedge \bigvee_2 \leq R \big\} \cup \big\{ \langle \text{varnothing} \rightarrow j \rangle; \langle j \rangle \in \mathcal{N}_t \big\} \end{aligned}$$

HetNet (`hetgat/utils.py`, abridged)

```
def build_hetgraph(pos, num_P=2, num_A=1, ..., with_two_state=False,
comm_range_P=-1, comm_range_A=-1):
    pos_coords = [cartesian_from_one_hot(x) for x in pos]      # decoded
    from observations
        for p1 in P_i:                                         # sender-
class range
    for x2 in range(num_P + num_A):
        if comm_range_P == -1 or comm_dist <= comm_range_P:
            (PnP if x2 < num_P else PnA).append([p1, x2])
    ...                                                         # same loop
for A senders
    data_dict[('P', 'p2p', 'P')] = (p2p_u, p2p_v)
    data_dict[('P', 'p2a', 'A')] = (p2a_u, p2a_v)
    data_dict[('A', 'a2p', 'P')] = (a2p_u, a2p_v)
    data_dict[('A', 'a2a', 'A')] = (a2a_u, a2a_v)
    data_dict[('P', 'p2s', 'state')] = (list(range(num_P)), [0] * num_P) #
P state node
    data_dict[('A', 'a2s', 'state')] = (list(range(num_A)), [1] * num_A) #
A state node
    data_dict[('state', 'in', 'state')] = ([0, 1], [0, 1])
    g = dgl.heterograph(data_dict, num_nodes_dict={'P': num_P, 'A': num_A,
'state': 2})
```

SoftRole (`softrole/graph.py`)

```

def build_role_graph(pos, active, comm_range=-1.0):
    n = len(pos)
    members = np.flatnonzero(active)
    src, dst = [], []
    for j in members:
        for k in members:
            if k != j and (comm_range < 0 or np.linalg.norm(pos[k] - pos[j])
<= comm_range):
                src.append(k); dst.append(j)
    n_real = len(src)
    src += [n] * len(members)          # node n is the null source: one
edge n -> j per receiver
    dst += members.tolist()
    g = dgl.heterograph({
        ('agent', 'comm', 'agent'): (tensor(src), tensor(dst)),
        ('agent', 'a2s', 'state'): (tensor(members), zeros(len(members))),
    }, num_nodes_dict={'agent': n + 1, 'state': 1})
    g.edges['comm'].data['null'] = torch.arange(len(src)) >= n_real
    return g

```

	HetNet	SoftRole	Why
Node types	P, A, state	agent, state	Heterogeneity moves from the graph's schema into the role
Agent relations	p2p, p2a, a2p, a2a, one attention per relation	comm, one attention	One softmax per receiver (guide, section 6)
Null slot	None	A source node n with one edge into each active receiver	Lets DGL's <code>edge_softmax</code> include the null term in the same normalization, numerically stable
Team size	Fixed at construction (<code>num_P</code> , <code>num_A</code>)	Any $ \mathbb{T} $, rebuilt every step	Variable teams need no new model
Membership	Everyone, always	<code>active</code> mask	Agents outside $\mathbb{T}$ get no edges, so they neither send nor receive

	HetNet	SoftRole	Why
State nodes	One per class, 3 relations	One SEN, one relation	Guide, section 7
Positions	Decoded from each agent's observation	True positions from the environment	See below

A latent issue in HetNet's positions. `batch_select_action_universal` passes `p[:dim2]` of each flattened observation, which is the cell-index one-hot of the *top-left* cell of the agent's $(2v+1)^2$ window, not of the agent's own cell. When that cell lies in the padding (agent within v cells of the top or left edge), the slice is all zeros and decodes to $(0, 0)$. Over 2,000 PCP resets (5×5 grid, $v = 2$, 2 predators and 1 capture agent), the decoded pairwise distances differed from the true ones in 1,930. It has no effect with `--comm_range_P -1 --comm_range_A -1`, the setting your reproduction uses, because range is then ignored. It matters the moment range is finite, so SoftRole reads `predator_loc` and `predator_capture_loc` from the environment instead.

The range here is one value for everyone. HetNet's is per sender class. If a later experiment needs per-agent ranges, make range part of k and test `norm ≤ range[k]`, the sender's range, as HetNet does.

3. The communication round

This is where the two methods differ most. HetNet's layer looks its weights up by class and edge type; `RoleGATLayer` mixes them with the gate. The equations are section 6 of the architecture guide.

HetNet (`hetgat/graph/fastbinary.py`, `HeteroGATLayerBinary`, abridged to the P→P path)

```

self.fc      = nn.ModuleDict({'P': Linear(in_P, out_P * H), 'A': ..., 'p2s':
..., 'a2s': ..., 'in': ...})
self.encoder = nn.ModuleDict({'P': Linear(in_P, msg_dim), 'A': Linear(in_A,
msg_dim)})
self.decoder = nn.ModuleDict({'P': Linear(msg_dim, out_P * H), 'A':
Linear(msg_dim, out_A * H)})
self.binarize = nn.Linear(1, 2)
self.p2p_src = Parameter((1, H, out_P)); self.p2p_dst = Parameter((1, H,
out_P))    # one pair per relation

Whp  = self.fc['P'](feat_dict['P']).view(-1, H, out_P)                #
self transform, class P
Wehp = self.encoder['P'](feat_dict['P'])                              #
logits, class P
msg_P = F.gumbel_softmax(self.binarize(Wehp.unsqueeze(-1)), tau=1, hard=True)
@ self.bin
m_p2p = self.decoder['P'](msg_P).view(-1, H, out_P)                  #
decoder of the receiving class
Attn_src = (m_p2p * self.p2p_src).sum(-1, keepdim=True)
Attn_dst = (Whp * self.p2p_dst).sum(-1, keepdim=True)
g['p2p'].apply_edges(fn.u_add_v('Attn_src_p2p', 'Attn_dst_p2p', 'e_p2p'))
e_p2p = self.leaky_relu(g['p2p'].edata.pop('e_p2p'))                #
additive score: static
g['p2p'].edata['a_p2p'] = edge_softmax(g['p2p'], e_p2p)              # one
softmax per relation
g['p2p'].update_all(fn.u_mul_e('msg_toP', 'a_p2p', 'm_p2p'), fn.sum('m_p2p',
'ft_p2p'))
Whp_new = Whp + ft_p2p + ft_a2p                                     #
weights sum to 2 with two classes

```

SoftRole (`softrole/bank.py`, `softrole/layers.py`)

```

class Bank(nn.Module):
    #  $W(g) = \sum_c g_c W_c$ ,  $b(g) = \sum_c g_c b_c$ 
    def experts(self, x):
        #  $(N, d_{in}) \rightarrow (N, K, d_{out})$ 
        return torch.einsum('ni,koi->nko', x, self.weight) + self.bias
    def forward(self, x, g):
        # mixing outputs = mixing
weights, for affine maps
        return torch.einsum('nk,nko->no', g, self.experts(x))

def straight_through_bits(u, nu, temperature=1.0):
    soft = torch.sigmoid((u + nu) / temperature)

```

```

        hard = (u + nu > 0).to(u.dtype)
        return hard + (soft - soft.detach())          # forward: exactly hard;
backward: d soft / du

# RoleGATLayer.forward(g, h, gate, nu=None, comm_off=False)
z = self.self_bank(h, gate).view(N, H, d)            #
own gate
u = self.enc(h, gate)                                #
sender's own gate
b = straight_through_bits(u, logistic_noise(u.shape, u) if nu is None else
nu, self.T)
src, dst = g.edges(etype='comm'); is_null = g.edges['comm'].data['null']
gd = gate[dst]                                        #
receiver's gate, per edge
dec_all = self.dec.experts(b).view(N, K, H, d)        #
each expert decodes each sender
m = einsum('ek,ekhd->ehd', gd, dec_all[src.clamp(max=N - 1)]) * (~is_null)[: ,
None, None]
pre = einsum('ek,ekhd->ehd', gd, self.att.experts(torch.cat([z[dst], m],
-1)))
e = (einsum('ek,khd->ehd', gd, self.a) * F.leaky_relu(pre, 0.2)).sum(-1) #
GATv2 score
e = torch.where(is_null[:, None], einsum('ek,kh->eh', gd, self.null), e) #
null score, receiver-only
alpha = edge_softmax(g['comm'], e)                    #
one softmax incl. null
if comm_off:
    alpha = is_null[:, None].to(h.dtype).expand_as(alpha) #
alpha_null = 1
M = copy_e_sum(g['comm'], alpha[..., None] * m)[:N]
out = F.relu(z + M).flatten(1) if self.merge == 'cat' else (z + M).mean(1)

```

Line by line

Guide equation	HetNet	SoftRole
$z_j = W^{\text{self}}(g_j) h_j + b^{\text{self}}(g_j)$	<code>fc[class]</code>	<code>self_bank(h, gate)</code>
$u_k = W^{\text{enc}}(g_k) h_k + b^{\text{enc}}(g_k)$	<code>encoder[class]</code> , then <code>binarize = Linear(1, 2)</code>	<code>enc(h, gate)</code> ; the 1→2 map is folded into the encoder

Guide equation	HetNet	SoftRole
$b = 1[u + v > 0], v \sim \text{Logistic}$	Two-way Gumbel-Softmax, hard, @ [0, 1]	<code>straight_through_bits</code>
$m^{jk} = W^{\text{dec}}(g_j) b_k + b^{\text{dec}}(g_j)$	<code>decoder[receiver class]</code>	Every expert decodes every sender once; each edge mixes them with the receiver's gate
$e_{jk} = a(g_j)^T \text{LeakyReLU}(W^{\text{att}}(g_j)[z_j \parallel m^{jk}])$	$\text{LeakyReLU}(a_{\text{src}}^T m + a_{\text{dst}}^T W h)$ per relation	Banked W^{att} per head, then the banked a
$e_j \oslash = w^{\oslash}(g_j)$	None	Banked scalar per head, on the null edge
One softmax over $N_t(j) \cup \{\oslash\}$	One softmax per relation	One <code>edge_softmax</code> over the comm relation, null edge included
$h^{(\ell+1)} = \text{ReLU}(z + M)$, last round $z + M$	Same, <code>cat</code> then <code>avg</code>	Same

Why it is written this way

- **Receiver-gated maps are mixed per edge.** The decoder's input (the bits) belongs to the sender and its gate to the receiver, so neither can be mixed per node. Running the K experts once per sender, $O(N K H d)$, then gathering and mixing per edge, $O(|\mathcal{E}| K H d)$, avoids building any per-edge matrix. W^{att} sees the per-edge input $[z_j \parallel m^{jk}]$, so its experts run per edge; with at most $N(N-1) + N$ edges this is negligible here.
- **The channel is not a change.** With the same Gumbel draws, HetNet's binarizer and `straight_through_bits` produce identical bits and gradients equal to 10^{-12} (test 2). HetNet's `Linear(1, 2)` only adds a slope and offset, $P(b = 1) = \sigma(\kappa z + \kappa_0)$, which the encoder's own weights and bias already supply.
- **Why `hard + (soft - soft.detach())` and not `hard + soft - soft.detach()`.** In floating point, $1 + s - s$ need not equal 1. The first version of this code sent some bits one rounding unit away from 0 or 1, which test 1 caught. Grouping the difference makes the forward value exactly `hard`.
- **The null edge leaks no gradient.** Its source index is clamped to a real node and its message multiplied by 0, and `torch.where` passes no gradient to the unselected score, so no agent's bits or decoder receive gradient from a null edge.

- **Communication off is exact.** Forcing α onto the null edge gives $M = 0$ for every receiver. Test 5 checks that this reproduces, to 10^{-12} , the logits of a graph with no edges at all.
- **Initialization.** Each expert, and each head of each expert, gets its own Xavier draw. With identical experts, $\partial\mathcal{L}/\partial g_c$ is the same for every c and the softmax Jacobian sends it to zero (test 9 measures a gradient below 10^{-12} on G). The attention vectors use HetNet's Xavier-normal with ReLU gain. Null scores start at 0; at initialization $\alpha_\emptyset$ averaged 0.33 in the PCP team, where each receiver has two neighbours.
- **The SEN reads, never writes.** `forward_sen` takes the same round input h_j with $sg(p_j)$ and κ_j , using HetNet's additive attention form, since it only feeds the critic. No tensor flows from it to an agent.

The banked rounds hold 67,744 parameters, against 13,040 in HetNet-Binary's agent-side layers: 4 weight sets instead of 2 classes, plus GATv2's attention matrices (16,896 of the total), which additive attention does not have.

4. The network

`SoftRoleNet` replaces `UAVNetA2CEasy`: one encoder and one LSTM for every agent instead of per-class paths, a role and gate between memory and communication, a shared masked output layer, and a per-agent critic. Sections 2–8 of the architecture guide give the equations.

HetNet (`hetgat/uavnet.py`, `UAVNetA2CEasy.forward`, abridged)

```

x_per_stat, x_act_stat = self.remove_excess_action_features_from_all(x)      #
position part only
x_per_obs = self.get_obs_features(x)                                         #
class channels, P only
state_per_stat = self.relu(self.prepro_stat(x_per_stat))                     #
Linear(625, 625)
h_ps, c_ps = self.f_module_stat(state_per_stat, (h_ps.double(),
c_ps.double()))
x_per_obs = self.relu(self.prepro_obs(x_per_obs))                           #
Linear(100, 100)
h_po, c_po = self.f_module_obs(x_per_obs, (h_po.double(), c_po.double()))
feat_dict['P'] = torch.cat([h_ps, h_po], dim=1)                              #
25 + 4
feat_dict['A'] = self.f_module_stat(self.relu(self.prepro_stat(x_act_stat)),
(h_as, c_as))[0]
feat_dict['state'] = torch.tensor([[self.num_P, self.num_A, self.world_dim,
500]] * 2) # constant
h1 = self.layer1(g, feat_dict)
h2 = self.layer2(g, h1)                                                       # h2['P'] (5) and
h2['A'] (6) ARE the logits
P_critic_value = self.P_critic_head(self.relu(h2['state'][:1]))
A_critic_value = self.A_critic_head(self.relu(h2['state'][1:]))

```

SoftRole (softrole/net.py, SoftRoleNet.forward, abridged)

```

x = F.relu(self.pre(self.encode_obs(obs, kappa)))                           #
[cells ; channels * sense ; sense]
inp = torch.cat([x, kappa, mem['u_bar'], F.one_hot(mem['a_prev'], 6)], 1)
H, C = self.lstm(inp, (mem['H'], mem['C']))
H, C = H * act, C * act                                                       #
agents outside N_t keep zero state

mu, s = self.role_enc(torch.cat([H, kappa], 1)).chunk(2, dim=-1)
var = self.var_floor + F.softplus(s)                                         #
sigma^2 = sigma^2_min + softplus
rho = mu + var.sqrt() * eps                                                  # one
reparameterized sample per step
gate = torch.softmax(self.gate(rho), dim=-1)                                # g =
softmax(G rho + g0)

same = mem['alive_prev'] & (kappa == mem['kappa_prev']).all(1)
lam = torch.where(same, lambda_rho, 0.0)[: , None]                          # reset

```

```

on spawn or capability change
prior_mean = lam * mem['rho_prev'] + (1 - lam) * self.eta(kappa_index(kappa))
prior_var = 1 - lam ** 2
kl = 0.5 * ((var + (mu - prior_mean) ** 2) / prior_var - 1 +
torch.log(prior_var / var)).sum(-1)

hs = torch.stack([n_active, sum_sense, sum_cap, grid_side])[None]    # phi_t,
from the current team
h = H
for layer in self.rounds:
    hs = layer.forward_sen(g, h, hs, rho, kappa)                    #
critic side, reads only
    h, b, a_null, _ = layer(g, h, gate, nu=noise.get(f'nu{1}'),
comm_off=comm_off)
logits = self.policy_head(h).masked_fill(no_capture & is_capture_action,
float('-inf'))
value = self.critic(torch.cat([hs.expand(n, -1), rho.detach(), kappa],
1)).squeeze(-1)
new_mem = {'H': H, 'C': C, 'u_bar': h * act, 'rho_prev': rho, 'kappa_prev':
kappa, 'alive_prev': active, ...}

```

What changed, and the equation behind it

Part	HetNet	SoftRole	Guide
Observation	P: position cells to one LSTM, class channels to another. A: position cells only, channels dropped	All 725 native inputs, channels multiplied by the sense flag, plus the flag: 726 inputs	§1, observations
Memory input	Own observation only	Observation, κ , $\bar{u}_{(t-1)} = h^{(L)}_{(t-1)}$, one-hot $a_{(t-1)}$	§2
Memory state	Three LSTM states sized by class counts fixed at construction	One dict of tensors, created per episode by <code>init_memory(n)</code>	§2, membership
Role	None	$q(p \mid H, \kappa)$, variance floor, one sample per step	§3
Weights	Chosen by class	Mixed by $g = \text{softmax}(G_p + g_o)$	§3-4

Part	HetNet	SoftRole	Guide
Output	The second GAT layer's output is the logits (5 or 6)	The second round outputs 16 features, then one shared layer to 6 logits, masked	§7
SEN input	The constant [num_P, num_A, world_dim, 500]	$\phi_t = [\text{SEN}_t , \Sigma \text{ sense}, \Sigma \text{ cap}, \text{grid side}]$	§7
Critic	One value per class from its state node	One value per agent from $[h_s; \text{sg}(\rho_j); \kappa_j]$	§7
Prior and rate	None	Closed-form KL to the persistence prior, per agent-step	§8–9

The output layer is separate for a reason: $\bar{u}$ must be features the next step can read, and HetNet's final layer emits logits of a class-specific size. Adding one 16→6 layer costs 102 parameters.

Details that matter

- **Encoding blindness.** The environment marks a blind agent with a -1 fill of its class channels. `encode_obs` multiplies the channels by the sense flag, turning the fill into 0, and appends the flag, so “no sensor” and “nothing seen” stay distinguishable. This is $x_{(j,t)}$ of the guide, §1, computed from the environment's own convention.
- **Masking by** `masked_fill(-inf)`. This is $\log 1[a \in \text{DONT_FILL}(\kappa)]$ added to the logits. `Categorical` renormalizes over the finite logits, so a masked action has probability exactly 0 (test 6 samples 5,000 steps without one). HetNet's 5-way predator head is recovered exactly, since the environment treats a predator's capture as a no-op.
- **The prior is vectorized over agents.** λ is chosen per agent from two booleans, alive at $t-1$ and κ unchanged, so a spawn and a capability change reset the same way. `rho_prev` is the previous sample itself, not detached, so the rate trains the previous step's encoder output within the BPTT window (guide, §8).
- **Baselines are modes, not models.** `constant` sets $p = 0$ (so $g = \text{softmax}(g_0)$, shared weights plus κ); `deterministic` uses $p = \mu(H, \kappa)$ with no noise and no rate; `capability` feeds zeros in place of H to the role encoder. The rate is set to 0 in these modes. All four modes have 233,513 parameters.

- **Interventions are arguments.** `comm_off=True` forces $\alpha_{\emptyset} = 1$; `rho_override` pins chosen agents' roles (the role-freeze test); `noise` fixes ε and ν , which the equivariance and channel tests use and which PPO would need (guide, §10).
- **Precision.** `main.py` makes float64 the default, and every SoftRole tensor inherits it. HetNet casts its hidden states with `.double()` explicitly; SoftRole needs no casts.

Parameters

Component	HetNet-Binary	SoftRole
Observation encoders	401,350 (a 625×625 layer holds 391,250)	93,056
Memory	66,896 (two LSTMs)	55,808
Role encoder, gate, η	none	2,370
Communication, agent side	13,040	67,744
Output layer	inside the second round	102
SEN and critic	10,546	14,433
Total	491,832	233,513

Counted from the instantiated models with the reproduction recipe's settings (5×5 grid, vision 2, 4 heads, 16 bits). The architecture guide first estimated the banks at roughly twice HetNet's communication parameters; the measured ratio is 5.2, and that guide now gives these figures.

5. Loss and update

The update keeps HetNet's structure (per-agent GAE, the ×50 policy weight, per-episode aggregation, clipping at 0.75) and changes four things: per-agent critics, normalization over valid entries only, the rate term, and gradient routing. The objective is the guide's §9:

$$\begin{aligned} \mathcal{L} = & c_{\pi} \big(\mathcal{L}_{\pi} + \beta \mathcal{L}_{\text{rate}} \big) + \mathcal{L}_V, \quad \mathcal{L}_{\pi} = -\frac{1}{E} \sum_{e,t,j} \log \pi(a_{j,t} | \hat{A}_{j,t}), \\ \mathcal{L}_{\text{rate}} = & \frac{1}{E} \sum_{e,t,j} D_{\text{KL}}(q_{j,t} | \text{Vert } p_{j,t}), \quad \mathcal{L}_V = \frac{1}{E} \sum_{e,j} \text{mean}_t (V_{j,t} - R_{j,t})^2 \end{aligned}$$

HetNet (`hetgat/policy.py`, `A2CPolicy.batch_finish_per_class`, abridged)

```

batch_advs = torch.zeros(batch_size, num_agents, sim_time)                #
padded to max_steps with 0
for i_b in range(batch_size):
    for j in range(num_agents):
        batch_returns[i_b][j][:r_size], batch_advs[i_b][j][:r_size] =
self.batch_GAE_per_class(i_b, j, num_P, num_A)
P_critic_target = torch.mean(batch_returns[:, :num_P, :], dim=1)          #
class-mean lambda-return
P_advs_norm = (P_advs - P_advs.mean()) / (P_advs.std() + eps)            #
statistics include the padding
for i_b in range(batch_size):
    batch_total_policy_loss.append(torch.sum(-P_log_prob_list * P_adv_n))
    batch_total_critic_loss.append(F.mse_loss(P_critic_list,
P_critic_target[i_b][:r_size]))
self.optimizer.zero_grad(set_to_none=False)
total_loss = total_policy_loss * 50 + total_critic_loss                    # both
/ batch_size (episodes)
total_loss.backward()
nn.utils.clip_grad_norm_(self.model.parameters(), self.max_grad_norm)    # 0.75
# self.optimizer.step()    <- commented out: Trainer's RMSprop steps after
grad /= num_steps

```

SoftRole (`softrole/policy.py`, `SoftRolePolicy.finish_batch`, abridged)

```

for ep in self.episodes:
    V = torch.stack(ep['value'])                                           # (T,
N), per-agent critic
    A = self.gae(torch.stack(ep['reward']), V.detach())                   # last
step bootstraps 0, as HetNet
    per_ep.append((ep, V, A, A + V.detach(), torch.stack(ep['active'])))
for flag in (False, True):                                              #
capture-capability groups
    vals = torch.cat([A[m & (ep['cap'] == flag)[None]] for ep, _, A, _, m in
per_ep])
    norm[flag] = (vals.mean(), vals.std()) if vals.numel() > 1 else (0.0,
1.0)
for ep, V, A, R, m in per_ep:
    A_n = (A - mean_of_group) / (std_of_group + eps)
    l_pi = l_pi - (torch.stack(ep['logp']) * A_n * m).sum()
    l_rate = l_rate + (torch.stack(ep['kl']) * m).sum()
    l_v = l_v + (((V - R) ** 2 * m).sum(0) / m.sum(0).clamp(min=1)).sum()

```

```

# sum_j mean_t
l_pi, l_rate, l_v = l_pi / E, l_rate / E, l_v / E
for p in self.model.parameters():
    if p.grad is None:
        p.grad = torch.zeros_like(p) #
allocate once, then only zero in place
    p.grad.zero_()
(self.c_pi * (l_pi + self.beta * l_rate)).backward(retain_graph=True)
l_v.backward(inputs=self.non_role_params) #
critic gradient kept off role params
nn.utils.clip_grad_norm_(self.model.parameters(), self.max_grad_norm)

```

Differences, each with its reason

	HetNet	SoftRole	Reason
Critic target	Class mean of agents' λ -returns	Agent's own λ -return $R = \hat{A} + V$	Classes are gone; rewards are per agent in PCP
Normalization groups	P and A	Capture flag 0 and 1	Same split on the fixed team; defined on any team
Normalization statistics	Over the padded (episodes $\times$ agents $\times$ max_steps) tensor, zeros included	Over valid agent-steps only	Padding zeros shrink the mean and change the scale by an amount that depends on how early episodes end, which differs between methods
Rate term	None	$\beta \cdot \Sigma \text{KL}$, same aggregation as $\mathcal{L}_\pi$	β keeps one meaning for any team size (guide, §9)
Gradient of $\mathcal{L}_V$	All parameters	Non-role parameters only	Roles follow ∇J_β and nothing else (guide, §9)
Inactive agents	Not possible	Masked out of all three sums	Σ over $j \in \mathbb{M}_t$ in the objective

Everything else is kept on purpose: $\gamma = 1$, $\lambda_{\text{GAE}} = 0.95$, $c_{\pi} = 50$, the per-episode division, clipping before the trainer divides by `num_steps`, and `RMSprop( $\alpha = 0.97$ ,  $\epsilon = 10^{-6}$ )`.

Why `backward(inputs=...)`

`l_v.backward(inputs=self.non_role_params)` accumulates $\partial \mathcal{L}_V / \partial \theta$ into `.grad` for the listed tensors only, while still traversing every path to them, including paths through p . The role encoder and G then receive exactly $c_{\pi} \nabla(\mathcal{L}_{\pi} + \beta \mathcal{L}_{\text{rate}})$. Two alternatives are wrong: detaching p inside the rounds would also cut the policy's pathwise role gradient, and backpropagating the sum and then zeroing the role gradients would erase their actor gradient too. Test 12 runs an update with $c_{\pi} = 0$ and checks that every role gradient is exactly zero while the LSTM and critic gradients are not.

Why gradients are allocated once and zeroed in place

`MultiProcessTrainer.obtain_grad_pointers` stores the `.grad` tensors of the main process and of every worker **by position**, once, at the first update, and later sums through those stored references. Three consequences follow:

1. A gradient that is `None` at the first update is left out of that process's list for good. If two processes disagree on which gradients were `None`, their lists are added position by position across different parameters.
2. A `.grad` that is ever *replaced* rather than modified in place leaves the stored reference pointing at the old tensor, so the sum uses stale values.
3. Spawned workers unpickle parameters without `.grad`.

So every gradient is allocated on the first call in each process, zeroed with `zero_()` afterwards, and written only by autograd's in-place accumulation. Test 12 checks that storage pointers are unchanged across two updates, and your probe (section 9) checks the full multiprocessing sum.

One inherited approximation, kept for comparability

Both methods bootstrap the last step with $V = 0$, whether the episode ended in success or was cut at `max_steps`. With $\gamma = 1$ and no time input, the critic cannot tell how many steps remain, so this adds variance to V (Pardo et al., 2018). It is kept because changing it for one method only would be a confound. If you fix it, fix it in both.

6. Rollout plumbing

`SoftRoleTrainer` subclasses `Trainer` and overrides two methods, `get_episode` and `compute_grad`. `run_batch`, `train_batch` and `MultiProcessTrainer` are inherited, so episode collection until `batch_size` steps, the division of gradients by `num_steps`, and the RMSprop step are literally HetNet's code.

HetNet (`trainer.py`, `Trainer.get_episode`, the `hetgat_a2c` branch, abridged)

```
state = self.env.reset(epoch)
for t in range(self.args.max_steps):
    if t == 0:
        prev_hid = self.policy_net.init_hidden(batch_size=state.shape[0])
# sizes from num_P, num_A
    x = [state, prev_hid]
    action_out, value, prev_hid =
self.policy.batch_select_action_universal(x, self.stats['num_episodes'])
    if (t + 1) % self.args.detach_gap == 0:
        prev_hid['P_s'] = (prev_hid['P_s'][0].detach(), prev_hid['P_s']
[1].detach())
        prev_hid['P_o'] = (prev_hid['P_o'][0].detach(), prev_hid['P_o']
[1].detach())
        prev_hid['A_s'] = (prev_hid['A_s'][0].detach(), prev_hid['A_s']
[1].detach())
    action = select_action(self.args, action_out)          # loops over
args.nfriendly_P, nfriendly_A
    action, actual = translate_action(self.args, self.env, action)
    next_state, reward, done, info = self.env.step(actual)
    self.policy.batch_rewards[self.stats['num_episodes']].append(reward)
    self.policy.append_log_probs_properly(actual)          # rebuilds
Categorical per class
```

SoftRole (`softrole/trainer.py`, `SoftRoleTrainer.get_episode`, abridged)

```

n_p, n_a = sr_env.sample_composition(args.sr_p_range, args.sr_a_range)
sr_env.set_composition(env, n_p, n_a) # before reset
state = self.env.reset(epoch)
kappa = sr_env.capabilities(env)
mem = model.init_memory(n_p + n_a)
self.policy.begin_episode(kappa[:, 1] > 0)
for t in range(args.max_steps):
    kappa = self._event(t, kappa, n_p) # phase 4: a
    predator may lose its sensor
    obs = sr_env.apply_sensor_mask(state[0].numpy(), kappa, base, n_sq)
    g = build_role_graph(sr_env.positions(env), active.numpy(),
args.sr_comm_range)
    out = model(torch.as_tensor(obs), torch.as_tensor(kappa), mem, active, g,
grid_side=args.dim)
    dist = Categorical(logits=out['logits'])
    a = dist.sample()
    next_state, reward, done, info = self.env.step([a.numpy()])
    self.policy.record(dist.log_prob(a), out['value'], out['kl'], reward,
active)
    mem = out['memory']
    mem['a_prev'] = a
    if (t + 1) % args.detach_gap == 0:
        mem = model.detach_memory(mem) # H, C, u_bar and
rho_prev together

```

Concern	HetNet	SoftRole
Team	Fixed by <code>--nfriendly_P/A</code> for the whole run	Drawn per episode, then written into the environment before <code>reset</code>
Carried state	Three LSTM tuples	H, C, $\bar{u}$, previous action, previous role, previous κ , previous membership
Truncation	Cuts the three LSTM tuples	Cuts every floating tensor in the memory at once
Sampling	<code>select_action</code> , per class, in <code>action_utils.py</code>	One masked <code>Categorical</code> over all agents
Log-probabilities	Recomputed from stored logits after the step	Taken from the same distribution that sampled

Concern	HetNet	SoftRole
Buffers	Indexed by <code>stats['num_episodes']</code>	A list of episode dicts, cleared after each update

Three details that are easy to get wrong

- **Truncate $\bar{u}$ and $p_{(t-1)}$ with H and C.** Both are recurrent state. If only H and C are cut, the graph keeps growing through $\bar{u}$ for the whole episode, memory rises with episode length, and gradient terms longer than the window survive for $\bar{u}$ alone. `detach_memory` detaches every floating tensor in the memory dict, so nothing can be forgotten when a field is added. Test 13 checks it.
- **Draw random team events from numpy's global stream.** `MultiProcessWorker.run` re-seeds torch, numpy and `random` with seed + id + 1 in each worker. A generator created in `SoftRoleTrainer.__init__` would be built once in the parent and copied into every worker, so all four processes would draw the same sequence of teams. My first version did exactly that; the environment's own position sampling already uses the global stream.
- **Set attributes on the unwrapped environment.** `data.init` returns `GymWrapper(gym.make(...))`. Reads fall through gym's wrappers, but an assignment such as `env.npredator = 3` on a wrapper creates an attribute on the wrapper and never reaches the environment. `raw_env` returns `wrapper.env.unwrapped`, and `set_composition` also updates `nfriendly_P`, which `_get_reward` uses as an index offset for capture agents.

`action_utils.py` and `multi_processing.py` are untouched. `select_action` is simply not called on this path, because it assumes the fixed class counts in `args`.

7. Environment changes by phase

`PredatorCaptureEnv` itself is never edited. Everything SoftRole needs is either already in the environment's state or can be imposed from outside it, in `softrole/env.py`:

Phase	Needs	How	Status
Fixed PCP team	κ and true positions	κ from agent order: predators (1, 0), capture agents (0, 1), since the env makes capture agents blind; positions from <code>predator_loc</code> and <code>predator_capture_loc</code>	Implemented, trained
Team drawn per episode	Counts changed before <code>reset</code>	<code>set_composition</code> writes six attributes on the unwrapped env	Implemented, trained
Held-out teams	Same, at evaluation only	<code>evaluate(..., n_p, n_a)</code>	Implemented
Sensor loss in an episode	An agent's observation without its sensor	<code>apply_sensor_mask</code> : the env's own <code>-1</code> fill, applied to the agent's class channels	Implemented, trained
Spawns and departures	A roster whose members change mid-episode	Not possible from outside the env; see below	Model side tested; env not implemented

Why sensor loss can be imposed from outside

In PCP, sensing affects only what an agent observes. No transition or reward reads it: movement, capture and rewards depend on positions and actions alone. So the environment with a failed sensor has the same dynamics, and its observation is a fixed function of the true observation and κ :

$$o^{\{\text{blind}\}}_{j,t} = \big[\backslash, \text{cells}_{j,t} \backslash ; ; \backslash ; -\mathbf{1} \backslash, \big] \\ \quad \text{when } \kappa^{\{\text{sense}\}}_{j,t} = 0$$

That is exactly what the env already does to capture agents (`action_blind`), so a blinded predator looks like a capture agent's view, and the transform is exact rather than an approximation. At the step where κ changes, the prior resets (guide, §8).

Why spawns need the environment

Three things in the env assume everyone is present from the first step to the last:

1. **Success.** `_get_reward` ends the episode only when every predator has reached the prey and every capture agent has captured. An agent that has not spawned yet can never do either.
2. **Occupancy.** `_get_cordinates` places all agents at `reset` ; an absent agent would still occupy a cell and appear in others' observations.
3. **Rewards.** The reward vector has one entry per rostered agent, including those absent.

A roster version of the env would carry an `active` array and skip absent agents in placement, observation, reward and the success test. The model already handles membership: agents outside `self.get` get no edges, keep zero state, and are masked out of all losses. Test 14 checks the strongest form of this: adding an inactive agent to the roster leaves every active agent's logits, values and rates unchanged to 10^{-10} .

Your README states that no sensor degradation, capability loss or within-episode membership experiment is in the current plan. The last two rows are therefore additions to that plan, not part of it.

Constraints to respect

- `self.get` + number of prey $\leq \text{dim}^2$, because `_get_cordinates` samples distinct cells without replacement.
- Episodes end only on full success or at `max_steps` , so larger teams have longer episodes on average; compare methods on the same team distribution.
- With teams of different sizes, `reward_per_agent` in `metrics.jsonl` averages by slot, and slot 2 is a predator in one episode and a capture agent in another. Use per-group returns for analysis (section 8).

8. Logging and diagnostics

`TrainingRecorder` gains five lines: any key in a batch's stats that starts with `sr_` is summed and written to `metrics.jsonl` as a per-episode mean. HetNet's rows are unchanged because HetNet emits no such key. Each diagnostic answers a question the architecture guide says must be monitored:

Key	Quantity	Question it answers
<code>sr_rate</code>	Mean $KL(q \parallel p)$ per agent-step, in nats	How much information a role carries beyond the previous role and κ (guide, §9); logged even at $\beta = 0$
<code>sr_gate_entropy</code>	Mean $H(g_{(j,t)})$, at most $\ln K = 1.386$	Do agents commit to a mixture, or stay uniform?
<code>sr_load_c</code>	Mean $g_{(j,t,c)}$ for each c	Expert collapse: one load near 1 means a shared-weight network (Shazeer et al., 2017)
<code>sr_alpha_null</code>	Mean $\alpha_{(j,\emptyset)}$, averaged over rounds	How much agents choose to listen; $1/(1 + N(j))$ is the uniform value
<code>sr_bits</code>	Bits sent per step by the team	Should equal $L \cdot n \cdot \bar{\mathbf{t}} $, the budget of the guide's §5
<code>sr_return_nocap</code> , <code>sr_return_cap</code>	Mean episode return per agent, by capture flag	Returns that stay comparable when team sizes vary
<code>sr_team_size</code>	Mean $ \mathbf{t} $	Checks the sampled team distribution

In the first epoch of the smoke run (random teams of 1–3 predators and 1–2 capture agents, untrained model), `sr_bits` was 85.3 with a mean team size of 2.67, which is 32×2.67 as the budget requires. `sr_gate_entropy` was 1.289 of a possible 1.386, loads were 0.18–0.30, and `sr_alpha_null` was 0.39. Those are the initial values training should move away from.

What to plot, and when

- **Rate against return, one point per β .** The β sweep traces a rate–return curve (guide, §9). A method whose return does not fall as rate is squeezed does not need its roles.
- **Rate over time within an episode, around an event.** With the persistence prior, rate should spike at a sensor loss for the affected agent and relax afterwards. A flat trace means the event did not change the role.
- **Loads over training.** Collapse usually shows early. Add a load-balancing penalty only if it happens (guide, §4).

The per-step traces for the event plots come from `softrole/evaluate.py`, not from training logs, so the logging cost of training stays one float per key per episode.

9. Tests

Each test turns one sentence of the architecture guide into an assertion. All 18 pass, along with the fork's 29, in 22 seconds on 2 CPUs (`pytest tests/`).

#	Test	Guide's claim	What it asserts
1	<code>bits_are_bernoulli_of_sigmoid</code>	§5: each bit is $\text{Bernoulli}(\sigma(u))$	Bits are exactly 0 or 1; over 200,000 draws each frequency is within 4 standard errors of $\sigma(u)$
2	<code>hetnet_gumbel_binarizer_is_the_same_channel</code>	§5: HetNet's binarizer computes the same thing	With the same Gumbel draws, HetNet's bits equal ours exactly and the gradients agree to 10^{-12}

#	Test	Guide's claim	What it asserts
3	<code>permutation_equivariance</code>	§6: the round is permutation-equivariant	Permuting 5 agents, their positions and their noise permutes logits, values and rates (range-limited graph)
4	<code>any_team_size_with_the_same_parameters</code>	§11: shared node functions run on any team	Teams (1,0), (0,1), (2,1), (4,2): correct shapes, finite outputs
5	<code>null_slot_isolated_agent_and_comm_off</code>	§6: empty neighbourhoods and communication-off are exact	Isolated agents get $\alpha_{\emptyset} = 1$; forcing $\alpha_{\emptyset} = 1$ on the full graph reproduces the isolated logits to 10^{-12}

#	Test	Guide's claim	What it asserts
6	<code>feasibility_mask</code>	§1, §7: no role grants a missing action	Capture logits are $-\infty$ for predators; 5,000 samples contain no predator capture
7	<code>no_path_from_sen_or_critic_into_actors</code>	§7, rule 1	The gradient of the logits with respect to every SEN and critic parameter is absent or zero
8	<code>role_is_learned_pathwise</code>	§10: roles learn through the reparameterized sample	$\nabla \log \pi$ reaches the role encoder and G
9	<code>identical_experts_block_all_gate_gradient</code>	§4: experts must differ at initialization	With tied experts the gradient on G is below 10^{-12} ; with Xavier-initialized experts it is not

#	Test	Guide's claim	What it asserts
10	<code>kl_closed_form_matches_monte_carlo</code>	§9: the closed-form rate	Closed form within 4 standard errors of a 400,000-sample estimate, for $\lambda = 0$ and 0.9
11	<code>prior_resets_when_capability_changes</code>	§8: $\lambda = 0$ on a capability change	The changed agent's rate equals the KL to $\pi(\eta(\kappa), l)$; unchanged agents' do not
12	<code>critic_gradient_never_reaches_role_parameters_and_storage_is_stable</code>	§9: $\mathcal{L}_V$ excluded from role parameters	With $c_\pi = 0$ every role gradient is exactly 0 and the LSTM's is not; <code>.grad</code> storage pointers survive two updates
13	<code>detach_cuts_every_recurrent_tensor</code>	§2: $\bar{u}$ is truncated with H and C	No tensor in the memory keeps a <code>grad_fn</code>

#	Test	Guide's claim	What it asserts
14	<code>inactive_agent_is_invisible_to_the_team</code>	§1: only  _t communicates	Adding an inactive agent to the roster changes no active agent's logits, value or rate (10^{-10})

Your gradient-contract probe, run on SoftRole

`tests/helpers/softrole_gradient_probe.py` imports your `hetnet_gradient_probe` and swaps two things: the policy factory, and the base class of its instrumented trainer, via `class ObservedSoftRoleTrainer(probe.ObservedTrainer, SoftRoleTrainer)`. Python's method resolution order then runs the probe's snapshots around SoftRole's own `compute_grad`, so no learner code is replaced. The probe's gates are unchanged: parameters shared after every step, the optimizer's gradient equal to the fresh sum of the four processes' gradients divided by `num_steps`, and cached gradient storage still current.

Actual HetNet gradient contract: PASS

update 1: shared_weights=True current_storage=True fresh_aggregation=True
max_gradient_error=0

update 2: shared_weights=True current_storage=True fresh_aggregation=True
max_gradient_error=0

update 3: shared_weights=True current_storage=True fresh_aggregation=True
max_gradient_error=0

(The banner text is the probe's own.) This is the test that would catch the pointer failures of section 5. It is worth adding to `tests/` as a pytest wrapper, like `test_hetnet_gradient_contract.py`.

Checks that are not unit tests

Check	Result
HetNet-Binary through the patched and the original <code>main.py</code> , same seed, 2 epochs	Identical <code>metrics.jsonl</code> rows (wall time aside) and identical <code>epoch_signatures.jsonl</code>
SoftRole through <code>main.py</code> , 1 and 2 processes (spawn)	Runs; metrics written; <code>sr_bits</code> = $L \cdot n \cdot \left\lceil \frac{L}{n} \right\rceil$
Random teams and a sensor loss at step 20	Runs
<code>evaluate</code> with each intervention	Runs, on an untrained model, so the numbers carry no meaning

What the tests do not cover

They check that the code computes the architecture as specified. They say nothing about whether it learns, and no training run here was long enough to tell. The first real evidence is Phase 1 of section 10.

10. Experiments and compute

The experiments must isolate the claim of the architecture guide (§12): a rate-limited adaptive role beats capacity-matched direct conditioning at frozen-policy recovery. Every baseline below is a flag on the same code, and HetNet enters only where it can run.

Phases

Phase	Training distribution	Methods	Question	Flags
0	Fixed PCP team (2, 1)	HetNet-Real, HetNet-Binary	Does the reproduction match the paper?	Your README plan, unchanged

Phase	Training distribution	Methods	Question	Flags
1	Fixed PCP team (2, 1)	HetNet-Binary; SoftRole at $\beta \in \{0, 10^{-3}, 10^{-2}, 10^{-1}\}$	Does SoftRole lose anything on HetNet's own task, at the same 32 bits per agent-step?	<code>--softrole --sr_beta β</code>
2	Teams drawn per episode: 1-3 predators, 1-2 capture agents	SoftRole in all four <code>--sr_role</code> modes	Does the gated role help across compositions, at equal capacity?	<code>--sr_p_range 1 3 --sr_a_range 1 2 --sr_role ...</code>
3	Phase 2 minus one team, e.g. (3, 2)	Same	Does it transfer, frozen, to the held-out team?	<code>evaluate(..., n_p=3, n_a=2)</code>
4	Phase 2 plus a sensor loss in each episode	Same	Does it recover after the event, frozen, and does the role cause it?	<code>--sr_sensor_loss_step</code> , then <code>evaluate</code> with interventions

HetNet cannot be trained across compositions, since its class counts and output sizes are fixed at construction. In Phases 2–3 it can serve as a per-composition specialist, trained on each team separately, which is a reference point, not a competitor.

What decides the claim

- **Full model against `deterministic`** (CASH-style direct conditioning on memory and κ through the same bank and the same n-bit channel). This is the comparison the claim is about.
- **Against `constant`** (shared weights plus κ): whether banking helps at all.
- **Against `capability`**: whether history adds anything beyond κ .
- **Interventions in Phase 4**, run by `softrole/evaluate.py` on frozen parameters: `freeze_role` pins the affected agent's role at its pre-event value while κ , masks and memory still update; `comm_off`; `role_mean`. If freezing the role does not reduce recovery, the role is not the mechanism, whatever the return.

The event step is fixed per run in this implementation. Randomizing it per episode, uniformly over a window, prevents the policy from timing the event, and is a one-line change in `SoftRoleTrainer._event`.

Statistics

At least 5 seeds per configuration, with the same seeds across methods so comparisons are paired. Report the interquartile mean over seeds with stratified bootstrap 95% confidence intervals (Agarwal et al., 2021), computed on the last 50 epochs of `success_rate` and per-group returns, plus every seed's curve. Fix the β grid and the evaluation protocol before looking at Phase 2 results.

Compute, measured

One update of 500 or more steps, single thread, float64, on this sandbox's 2.1 GHz Xeon:

Model	Steps	Rollout	Update	Per step	2,000 epochs × 10 updates
HetNet-Binary	561	18.5 s	6.1 s	0.044 s	about 127 h
SoftRole, team (2, 1)	522	5.7 s	4.4 s	0.019 s	about 56 h

The projection multiplies the per-step time by about 520 steps per update, 10 updates per epoch and 2,000 epochs; the 4 processes run in parallel, so they do not add time. The difference is mostly DGL overhead. In a profile of 212 HetNet steps, the two GAT layers took 5.3 of the 7.0 s spent choosing actions: every relation gets its own sequence of DGL calls, and each `g[ 'p2p' ]`-style lookup builds a new graph view (25,016 lookups). SoftRole's round makes one `edge_softmax` and one `copy_e_sum` call over a single relation. Teams of up to 5 agents will be slower than the (2, 1) figure; that was not measured.

This matters for Phase 0 now. Your Slurm default is 48 hours and the README says automatic resume is not implemented. On this CPU a 2,000-epoch HetNet run needs about 2.6 times that. Stokes CPUs may well be faster, so time one epoch there before sizing the array, and either request the measured time or add resume: `--load` restores the model, the optimizer and the log, but the recorder refuses to append to an existing `metrics.jsonl` and the epoch counter restarts at 0.

Slurm

One array task per (configuration, seed), the same 4 CPUs and 16 GB as the reproduction jobs, with `--time` set from the measured epoch time plus 20%. A short screening pass first, for example 500 epochs across the β grid in Phase 1, keeps full-length runs for the settings that survive.

11. Risks and pre-flight checklist

Risk	How it shows	Response
Roles go unused (q collapses to p)	<code>sr_rate</code> near 0; loads near uniform; <code>freeze_role</code> changes nothing	The guide's fallback objective (§11): make roles predict the agent's own future behaviour
Expert collapse	One <code>sr_load_c</code> near 1	A load-balancing penalty, only if observed (Shazeer et al., 2017)
The rate term dominates early	At initialization <code>sr_rate</code> was about 6.2 nats per agent-step, because the encoder starts near variance 0.7 ($0.01 + \text{softplus}(0)$) while the $\lambda_\rho = 0.9$ prior allows 0.19	Keep small β in the grid; if large β stalls learning, anneal β from 0 over the first epochs (Bowman et al., 2016) and report it
Straight-through bias	Not observable directly	Inherited from HetNet and identical in both methods; state it
Truncation treated as terminal	Noisy critic	Inherited; change both methods or neither (section 5)
Gradient aggregation errors	Silent	The probe of section 9, rerun after any change to parameters or the update
Finite communication range with HetNet	HetNet's graph built from decoded, often wrong positions	Pass true positions to <code>build_hetgraph</code> before any finite-range HetNet run (section 2)

Risk	How it shows	Response
Slot-wise rewards on variable teams	Misleading <code>reward_per_agent</code>	Analyse <code>sr_return_nocap</code> and <code>sr_return_cap</code> (section 8)
Wall time	Runs killed at 48 h with no resume	Time an epoch on Stokes first (section 10)

Checklist

- ☐ Apply `softrole.patch` on a new branch of your fork and run `pytest tests/` (47 tests).
- ☐ Run `tests/helpers/softrole_gradient_probe.py --output <file>` and confirm PASS; wrap it as a pytest test.
- ☐ Repeat the HetNet bit-identity check on Stokes: 2 epochs, same seed, original and patched `main.py`, compare `metrics.jsonl` and `epoch_signatures.jsonl`.
- ☐ Time one epoch of HetNet-Binary and of SoftRole on a Stokes node; size `--time`, and decide on resume.
- ☐ Phase 1 screening: β grid, 500 epochs, 3 seeds; keep the survivors for full runs.
- ☐ Write down the β grid, the seeds and the evaluation protocol before running Phase 2.
- ☐ Randomize the event step per episode before Phase 4.
- ☐ Before any run with a finite range, give HetNet true positions.

References

Companion guide: `SoftRole-HetNet: The Architecture, Explained`, for every equation cited as “guide, §n”.

Code

- Seraj et al. [Learning Efficient Diverse Communication for Cooperative Heterogeneous Teaming](#) (HetNet). AAMAS 2022. Code: [CORE-Robotics-Lab/HetNet](#); the excerpts above are from your fork of it.
- Wang et al. [Deep Graph Library](#). 2019.

Methods used in the implementation

- Brody, Alon and Yahav. [How Attentive are Graph Attention Networks?](#) (GATv2). ICLR 2022.
- Jang, Gu and Poole. [Categorical Reparameterization with Gumbel-Softmax](#). ICLR 2017.
- Maddison, Mnih and Teh. [The Concrete Distribution](#). ICLR 2017.
- Bengio, Léonard and Courville. [Estimating or Propagating Gradients Through Stochastic Neurons](#) (straight-through). 2013.
- Schulman et al. [High-Dimensional Continuous Control Using Generalized Advantage Estimation](#). ICLR 2016.
- Shazeer et al. [Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer](#). ICLR 2017.
- Pardo et al. [Time Limits in Reinforcement Learning](#). ICML 2018.
- Bowman et al. [Generating Sentences from a Continuous Space](#) (KL annealing). CoNLL 2016.

Evaluation

- Agarwal et al. [Deep Reinforcement Learning at the Edge of the Statistical Precipice](#) (interquartile mean, stratified bootstrap). NeurIPS 2021.
- Fu, Jain et al. [Capability-Aware Shared Hypernetworks for Flexible Heterogeneous Multi-Robot Coordination](#) (CASH, the direct-conditioning baseline). CoRL 2025.